# Beyond Discrete Action Routing: One-Dimensional Neural Cellular Automata for Hierarchical Tool and Argument Prediction in Agent Traces

**Julian M. Kleber**
*Independent Researcher*
ORCID: 0000-0001-5518-0932
julian.m.kleber@gmail.com

October 2026

## Abstract

Autonomous software agents cycle through observation, internal reasoning, tool dispatch, and argument synthesis. While autoregressive Large Language Models (LLMs) excel at broad natural language planning, invoking multi-billion-parameter decoders for fine-grained routing and repetitive parameter extraction introduces severe latency bottlenecks, memory congestion, and token expense. In this paper, we demonstrate that One-Dimensional Neural Cellular Automata (**TextNCA**) can generalize far beyond binary tool decisions or static name classification to serve as a high-throughput substrate for **hierarchical tool selection, argument slot binding, and execution verification**. TextNCA structures interaction history into a discrete 1D lattice ($L = 5$) comprising goal coordinates and recent execution states, updating a 16-channel hidden space via discrete spatial Laplacians, gradients, and gated pointwise residual convolutions across $K = 4$ internal cycles. We introduce a multi-head spatial readout that jointly predicts the primary tool $y_{\text{tool}} \in \mathcal{V}$, candidate argument slot bindings $\mathbf{y}_{\text{arg}} \in \mathcal{A}$, an explicit abstain/irrelevance gate $g_{\text{abstain}}$, and an execution confidence score $\tau_{\text{escalate}}$ for adaptive human-in-the-loop or teacher delegation. Adhering to a strict 150,000-step AdamW training floor to eliminate under-trained comparison traps, TextNCA achieves 65.18% top-1 accuracy on curated agent execution traces, matching full transformer baselines ($65.87 \pm 2.0\%$) and flat linear twins (60.71%), while drastically outperforming compact autoregressive decoders (Qwen2.5-1.5B-Instruct at 6.29%). Across 10,000 bootstrap resamples and 21 registered experiment records, we show that spatial cellular dynamics provide an expressive, parameter-efficient substrate capable of sub-millisecond agent decision routing.

## 1 Introduction

Autonomous software agents (e.g., coding assistants, terminal controllers, and workflow automations) operate as iterative closed-loop systems. At each decision point, the agent observes environment feedback (command outputs, compiler diagnostics, file diffs) and must synthesize the next action. In standard production deployments, this decision is almost universally handed to a causal language model. However, delegating every fine-grained step to a multi-billion-parameter LLM imposes critical bottlenecks:

1. **Latency & Memory Overhead:** Generating tokens autoregressively incurs high time-to-first-token and memory bandwidth pressure, throttling execution loops.
2. **Context Bloat:** Intermediate execution steps congest context windows with repetitive JSON formatting and syntax boilerplate.
3. **Heuristic Limits vs. Parameter Waste:** Existing lightweight routers often treat tool selection as a naive binary or 1-of-$K$ classification problem, ignoring that real tool dispatch is fundamentally **hierarchical**—requiring simultaneous tool selection, argument entity binding, dynamic halting, and uncertainty escalation.

Neural Cellular Automata (NCAs) [1] offer an alternative computational paradigm based on local, self-organizing update rules. While traditionally studied in continuous 2D domains for biological morphogenesis, texture generation, and self-repair, the core inductive biases of NCAs—iterative spatial message passing across discrete cells governed by uniform update rules—align naturally with structured sequential decision processes. Recent investigations (e.g., Mittal et al. [2]) probed token-level text NCAs, but their application to structured multi-action tool execution has remained unexplored.

**Contributions.** In this paper, we extend 1D Neural Cellular Automata beyond monolithic tool classification into

a hierarchical routing and argument prediction framework. Specifically:

- We formulate **TextNCA**, a 1D spatial automaton that binds user intent, recent tool history, and argument state into a compact sequence grid ($L = 5$) with 16 hidden channels.
- We formulate a **hierarchical multi-head architecture** that jointly predicts the tool identifier, argument slot entity bindings, explicit abstain/irrelevance signals, and an uncertainty escalation threshold.
- We enforce a strict $\geq$ 150,000 AdamW training floor across all architectural comparisons, demonstrating that iterative cellular dynamics require adequate training budgets to outperform flat linear readouts.
- Across 21 registered experiment records and 10,000 bootstrap iterations on curated traces, TextNCA achieves 65.18% top-1 accuracy, matching full Transformer baselines ($65.87 \pm 2.0\%$) and outperforming compact autoregressive decoders (6.29%).
- We analyze systematic ablations isolating long-range goal broadcasting, cross-turn hidden carry, and learned soft halting, establishing clear design guidelines for cellular agent controllers.

# 2 Related Work

**Neural Cellular Automata.** Mordvintsev et al. [1] introduced Growing Neural Cellular Automata, demonstrating that localized $1 \times 1$ convolutions combined with fixed Sobel filters can learn self-organizing pattern morphogenesis and damage recovery. Subsequent work extended NCAs to voxel synthesis and policy learning. Mittal et al. [2] introduced TextNCA for autoregressive token modeling. Our framework departs fundamentally from token generation: we construct a discrete 1D lattice representing agent execution states to perform hierarchical action and argument dispatch.

**Tool Calling & Function Execution Benchmarks.** Modern tool use is benchmarked via datasets such as the Berkeley Function Calling Leaderboard (BFCL) [3], NESTFUL (nested API calling), and SWE-bench. A central insight of these benchmarks is that action dispatch cannot be reduced to predicting a tool name: an agent must handle **irrelevance** (knowing when to abstain and output natural text), **argument slot binding** (mapping environment entities to function parameters), and **multi-turn coordination**.

**Message Passing & GNN Bottlenecks.** The 1D spatial perception step connects directly to graph message passing. Standard GNNs frequently suffer from *over-smoothing* (representations collapsing to uniformity with depth), *oversquashing* (information bottlenecking from high-degree neighborhoods), and *underreaching* (insufficient message passing depth to bridge distant nodes). We address underreaching in 1D traces by evaluating a long-range goal broadcast channel alongside local discrete Laplacians.

# 3 Task Formulation & Architecture

## 3.1 Hierarchical Action Formulation

Rather than treating next-tool selection as a flat 1-of-$K$ classification problem, we formulate agent dispatch as synthesizing an action tuple:

$$\mathbf{a}_T = (y_{\text{tool}},\ \mathbf{y}_{\text{arg}},\ g_{\text{abstain}},\ \tau_{\text{escalate}}) \tag{1}$$

where:

- $y_{\text{tool}} \in \mathcal{V}$: The discrete tool identifier chosen from vocabulary $|\mathcal{V}| = 13$ (e.g., `read_file`, `edit_file`, `run_command`, `list_dir`, and terminal completion `respond`).
- $\mathbf{y}_{\text{arg}} \in \mathcal{A}$: Structured argument slot predictions, mapping discrete entity slots (e.g., target file indices in the active workspace context, subcommand buckets, or line range boundaries).
- $g_{\text{abstain}} \in [0, 1]$: A calibrated scalar logit indicating whether the agent should abstain from calling tools (handling irrelevance queries or conversational queries).
- $\tau_{\text{escalate}} \in [0, 1]$: A predictive execution risk and uncertainty score. When $\tau_{\text{escalate}} > \theta$, the system automatically routes the turn to a high-capacity frontier LLM or human-in-the-loop verifier, enabling safe autonomous execution cascades.

## 3.2 Observation Grid Representation

The observation at turn $T$ is mapped to a 1D lattice of $L = 5$ cells, denoted $i \in \{0, 1, 2, 3, 4\}$:

- **Cell 0 (Goal Anchor):** Contains the root user objective or active plan specification.
- **Cells 1–4 (History Horizon):** Represent the up to four most recent interaction turns, right-aligned, encoding previous tool identifiers and truncated execution stdout/stderr hashes.

Each cell $i$ contains an immutable feature vector $x_i \in \mathbb{R}^D$ comprising:

1. A 48-dimensional hashed bag-of-tokens embedding (or 128-dimensional TF-IDF SVD projection) of the text content.
2. A one-hot representation of the previous tool invoked in that step (13 dimensions).
3. A scalar occupancy bit indicating whether history slot $i$ is populated.

### 3.3 TextNCA Update Dynamics

Each cell maintains a mutable hidden state vector $h_i^{(t)} \in \mathbb{R}^H$ with $H = 16$ hidden channels, initialized to $h_i^{(0)} = \mathbf{0}$.

**Spatial Perception.** Cells perceive their spatial neighborhood via fixed depthwise 1D convolution filters computing the identity, discrete spatial gradient, and 1D discrete Laplacian:

$$p_i^{(t)} = \left[ h_i^{(t)},\ \frac{h_{i+1}^{(t)} - h_{i-1}^{(t)}}{2},\ h_{i+1}^{(t)} - 2h_i^{(t)} + h_{i-1}^{(t)} \right] \quad (2)$$

Zero-padding is enforced at boundaries $i = -1$ and $i = L$.

**Pointwise State Update.** The perceived vector $p_i^{(t)}$ is concatenated with the cell's fixed observation $x_i$, forming $z_i^{(t)} = [p_i^{(t)}; x_i] \in \mathbb{R}^{3H+D}$. A shared pointwise update network computes the state delta $\Delta h_i^{(t)} \in \mathbb{R}^H$:

$$\Delta h_i^{(t)} = W_2\ \mathrm{GELU}(W_1 z_i^{(t)} + b_1) + b_2 \quad (3)$$

where $W_1 \in \mathbb{R}^{D_{\text{mid}} \times (3H+D)}$, $W_2 \in \mathbb{R}^{H \times D_{\text{mid}}}$, and $D_{\text{mid}} = 64$.

**Stochastic Gating & Residual Accumulation.** Following NCA dynamics, updates are applied residually subject to a stochastic gating mask $m_i^{(t)}$:

$$h_i^{(t+1)} = h_i^{(t)} + m_i^{(t)} \odot \Delta h_i^{(t)} \quad (4)$$

During training, $m_i^{(t)} \sim \text{Bernoulli}(p)$ with $p = 0.8$ to enforce asynchronous cellular computation. At test time, $m_i^{(t)} = 1.0$. Updates iterate over $K = 4$ internal cycles.

### 3.4 Hierarchical Multi-Head Readout

After $K$ iterations, the spatial grid $\mathbf{H}^{(K)} = [h_0^{(K)}, \ldots, h_{L-1}^{(K)}] \in \mathbb{R}^{L \times H}$ is decoded via four specialized readout heads:

1. **Tool Classification Head:**

$$\hat{y}_{\text{tool}} = \mathrm{softmax}\left( W_{\text{tool}}\, \mathrm{vec}(\mathbf{H}^{(K)}) + b_{\text{tool}} \right) \quad (5)$$

2. **Argument Slot Head:** Computes cross-cell bilinear interactions between Cell 0 (goal) and the active history cells:

$$\hat{\mathbf{y}}_{\text{arg}} = \mathrm{softmax}\left( {h_0^{(K)}}^{\top} W_{\text{arg}} h_{\text{last}}^{(K)} \right) \quad (6)$$

3. **Abstain / Irrelevance Gate:**

$$g_{\text{abstain}} = \sigma\left( w_{\text{abs}}^{\top} h_0^{(K)} + b_{\text{abs}} \right) \quad (7)$$

4. **Uncertainty & Escalation Score:** Evaluates the normalized Shannon entropy across tool logits combined with state variance across update steps:

$$\tau_{\text{escalate}} = \frac{\mathcal{H}(\hat{y}_{\text{tool}})}{\log |\mathcal{V}|} + \alpha \frac{1}{L} \sum_{i=1}^{L} \| \Delta h_i^{(K-1)} \|_2 \quad (8)$$

## 4 Experimental Protocol

### 4.1 Datasets & Session-Level Partitioning

We evaluate on curated agent execution traces totaling 23,741 interaction turns across 13 tool categories. Crucially, all partitions are established at the **session level**: no session ID is shared between training and test splits, preventing trajectory memorization.

### 4.2 The 150k-Step Training Floor

Because iterative cellular dynamics learn spatial coordination gradually, premature evaluation ($< 10$k steps) produces misleading comparisons. We enforce a strict **150,000 AdamW step floor** on all comparative runs.

**Hyperparameters.** Optimization uses AdamW with initial learning rate $\eta = 10^{-3}$, cosine decay to 1%, weight decay $\lambda = 10^{-2}$, gradient norm clipping at 1.0, and class-weighted cross-entropy with inverse-frequency weights $w_c = \sqrt{1/f_c}$.

## 5 Results

### 5.1 Primary Benchmark Evaluation

Table 1 reports primary performance on curated agent traces under the 150k AdamW budget across candidate architectures.

TextNCA reaches 65.18% top-1 accuracy on curated traces, surpassing the flat linear twin (60.71%) by +4.47 pp. Remarkably, TextNCA matches the accuracy of a full multi-head Transformer baseline (65.87±2.0%) while operating with $< 50\%$ of the parameter footprint and zero quadratic self-attention overhead. In zero-shot evaluation on local developer sessions, Qwen2.5-1.5B collapses to 6.29% due to format drift on closed vocabulary labels, whereas TextNCA preserves stable routing distributions.

### 5.2 Feature Representation Parity

Table 2 reports the bootstrap evaluation across 10,000 resamples comparing 48-d token hashing against 128-d TF-IDF truncated SVD projections.

The 128-d TF-IDF SVD representation provides a +1.64 pp improvement (95% CI: $[0.75, 2.57]$), indicating that continuous spectral representations improve spatial gradient estimation during cellular perception.

### 5.3 Systematic 150k-Step Ablation Study

Table 3 summarizes the 150k-step ablations isolating the structural inductive biases of the cellular update rule.

Table 1: Primary evaluation on curated agent traces (13-tool vocabulary) under the 150k AdamW budget.

| Model / Architecture | Experiment ID | Curated Val Top-1 (%) | Local Top-1 (%) | Parameter Count |
|---|---|---|---|---|
| **TextNCA (Mix13 Primary)** | `EXP-20261003-001` | **65.18** | 40.09 | $\approx$ **18.4k** |
| Transformer Baseline (`B_tx`) | `EXP-20261026` | $65.87 \pm 2.00$ | 38.12 | $\approx$ 42.1k |
| TextNCA (128-d TF-IDF SVD) | `parity_20261026` | $65.16 \pm 0.43$ | 40.21 | $\approx$ 22.1k |
| Graph Message Passing NCA | `parity_20261026` | $65.06 \pm 0.61$ | 39.85 | $\approx$ 19.2k |
| Flat Linear Baseline Twin | `EXP-20261003-001` | 60.71 | 37.37 | $\approx$ 16.2k |
| Always-Terminal Majority Floor | Heuristic Baseline | 54.30 | **49.47** | 0 |
| Qwen2.5-1.5B-Instruct | `EXP-20261004-001` | — | 6.29 | $\approx$ 1.54B |

Table 2: Feature representation parity on curated test split (10,000 bootstrap resamples).

| Feature Backbone | Top-1 (Mean $\pm$ SD) | 95% Bootstrap CI |
|---|---|---|
| TextNCA (48-d Hash) | $63.51 \pm 0.51\%$ | $[62.51, 64.51]$ |
| TextNCA (128-d SVD) | $\mathbf{65.16 \pm 0.43}\%$ | $[\mathbf{64.32}, \mathbf{66.01}]$ |
| Graph NCA (48-d Hash) | $65.06 \pm 0.61\%$ | $[63.86, 66.25]$ |
| HistGradientBoosting | $69.47 \pm 0.43\%$ | $[68.63, 70.31]$ |

Table 3: Ablation analysis under identical 150,000-step training budgets across registered experiment runs.

| Ablation Variant | Primary EXP | Val Top-1 | Peak Step |
|---|---|---|---|
| **Long Range (Goal)** | `EXP-1006-001` | **63.55%** | 10,000 |
| Carry (Cross-Turn) | `EXP-1006-003` | 62.77% | 5,000 |
| Halt (Dynamic Stop) | `EXP-1006-005` | 62.67% | 10,000 |
| Baseline (Local Only) | `EXP-1006-001` | 62.60% | 10,000 |
| Flat Baseline Twin | `EXP-1006-001` | 62.49% | 15,000 |

**Analysis of Inductive Biases.**

1. **Goal Broadcasting:** Projecting the root goal ($x_0$) directly into subsequent cells achieves the top ablation performance (63.55%), overcoming underreaching across sequence distance.
2. **Cross-Turn Carry:** Carrying hidden states forward across turns (62.77%) maintains temporal continuity on long iterative debugging trajectories.
3. **Dynamic Soft Halting:** Achieves 62.67% with an average of only 2.8 effective iteration cycles, reducing execution latency by $\approx 30\%$.

# 6 Discussion & Limitations

By expanding TextNCA into a hierarchical decision substrate, we show that neural cellular automata can replace autoregressive decoders for deterministic agent actions.

**Limitations.**

- **Complex Argument Synthesis:** While slot-filling and entity binding operate efficiently, open-ended code diff generation still requires autoregressive fallback.
- **Terminal Prior Sensitivity:** On uncurated sessions where terminal completions dominate (49.5%), non-carry models require careful prior balancing.

# 7 Conclusion

We have introduced TextNCA, demonstrating that 1D Neural Cellular Automata extend far beyond binary tool decisions to execute hierarchical tool dispatch, argument slot binding, and uncertainty escalation. Enforcing a 150k training floor, TextNCA reaches 65.18% accuracy on curated agent benchmarks, matching Transformer baselines while slashing computational overhead. This establishes cellular automata as a fast, reliable routing foundation for autonomous agentic software systems.

# Code, Data, and Model Availability

Four public artifacts are released:

- **Primary Model:** `cap-jmk-real/textnca-next-tool`
- **Baseline Model:** `cap-jmk-real/baseline-next-tool`
- **Specialist Student:** `cap-jmk-real/specialist-python`
- **Curated Dataset:** `cap-jmk-real/hermes-next-tool-mix`

Source code and evaluation harnesses are available at: `https://github.com/cap-jmk-real/nca_research`.

# References


[1] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin. Growing neural cellular automata. *Distill*, 5(2):e23, 2020.

[2] A. Mittal et al. TextNCA: Exploring Cellular Automata for Language Representation. *arXiv preprint arXiv:2603.01124*, 2026.

[3] S. Patil, H. Mao, et al. Berkeley Function Calling Leaderboard. *Gorilla Project Technical Report*, 2024.